01 / PROJECT OVERVIEW

PipelineMedic

An evidence-driven data reliability workbench

A data pipeline can complete without an exception and still publish incorrect numbers. PipelineMedic recreates that problem with real transactions and controlled failures, investigates the evidence, and tests a bounded repair before a human approves it.

In simple terms

Imagine a delivery truck accidentally unloading the same boxes twice. Counting the boxes tells you something is wrong; inspecting delivery labels explains why. PipelineMedic does the equivalent for data: it compares source identities, checks joins and transformations, and verifies the repaired result against trusted records.

What is built

- A working local dashboard with investigation, evidence, repair, history, evaluation, and dataset views.

- A real SQL pipeline, a trained novelty detector, a LangGraph investigator with deterministic and local-LLM modes, and an independent repair verifier.

- Persistent audit history, guarded approval and rollback, a reproducible fault benchmark, automated tests, and container/CI configuration.

Measured evidence

Evidence	Result
Source records	541,909
Controlled benchmark	120 cases / 96 faulty batches
Verified offline repairs	100% of injected faulty batches
Unnecessary offline repairs	0
Automated test suite	53/53 passing; 94.6% statement coverage

These are laboratory results for known fault families. They do not establish a production SLA, generalization to arbitrary pipelines, or independent LLM diagnostic accuracy.

Real data. Explicit semantics.

The source is UCI Online Retail, donated by Daqing Chen: transactions from a UK non-store retailer between December 1, 2010 and December 9, 2011. It contains 541,909 rows and 25,900 distinct invoice numbers across 38 countries. The dataset is CC BY 4.0; software is MIT licensed.

Source field	Interpretation / treatment
InvoiceNo	Invoice identifier; C-prefixed values indicate cancellations.
StockCode / Description	Product identifier and text; used to derive the lab dimension.
Quantity / UnitPrice	Signed quantity and GBP unit price; preserve negative quantities.
InvoiceDate	Historical batch partition and chronology.
CustomerID	Nullable; 135,080 missing values retained, not treated as defects.
Country	Used to demonstrate source-field fidelity.
source_row_id	Added positional provenance key, not a business key.

Why this dataset fits

It is large enough to make changes in counts and revenue meaningful, yet small enough for a laptop. Returns, incomplete customer identification, and seasonal changes create realistic counterexamples to simplistic cleaning. The derived product dimension enables join-cardinality failures without falsely claiming the original dataset contains multiple source tables.

Reproducible ingestion

The ingestion script downloads the original XLSX archive, verifies a pinned SHA-256 digest, retains every row, normalizes column names, assigns source-row identities, and writes compressed Parquet plus a manifest. A retry duplicate repeats the same provenance ID. Distinct source rows with identical business values remain distinct.

Citation: Chen, D. (2015). Online Retail. UCI Machine Learning Repository. https://doi.org/10.24432/C5BW33. Data source: https://archive.ics.uci.edu/dataset/352/online+retail

An inspectable path from source to action

```
XLSX -> verified Parquet landing
  -> ingestion adapter + derived product dimension
  -> DuckDB fact_sales SQL -> daily metrics
  -> data contracts + statistical novelty detector
  -> LangGraph decision / read-only probe loop
  -> allowlisted repairs on a copied snapshot
  -> output verification -> human approval -> rollback
```

Layer	Implementation and rationale
Analytics	DuckDB executes actual joins and materializes fact outputs.
Data science	pandas + scikit-learn: small reproducible model on temporal batches.
Investigation	LangGraph controls tool selection and bounded state transitions.
API	FastAPI + Pydantic validate inputs and expose typed endpoints.
Jobs / persistence	Two worker threads, a six-job limit, SQLite WAL and audit events.
UI	Semantic HTML, CSS, JavaScript, and SVG; no CDN or build service.
Delivery	Pinned dependencies, Docker, CI workflow, scripts, and test evidence.

Separation of responsibilities

Fault injection belongs to the experiment harness. The investigator receives only read-only probe results and observed alerts. Fault labels, seeds, severities, and expected diagnoses stay outside its state. The evaluator-only reference is never an agent tool. The API stores the selected scenario for the user, but the LLM does not see it.

The pipeline is deliberately explicit SQL. This version does not claim dbt, Dagster, Spark, Airflow, or external-warehouse integrations. Those would be adapter work, not prerequisites for proving the core investigation and verification behavior.

Novelty is a signal, not a diagnosis

Temporal protocol

Train on available daily batches through July 31, 2011; calibrate on August; evaluate selected dates from September through November. The partial final December day is excluded from the test period. Non-trading days are not invented as zero batches. No held-out batch enters model fitting or the calibration reference.

Features and model

Six batch features: row count, total absolute revenue, mean price, mean absolute quantity, missing-customer rate, and return rate. Nonnegative features receive log1p transformation. Isolation Forest uses 150 trees, random seed 73, one worker, and no supervised fault labels.

```
novelty(x) = - model.score_samples(x)
tail_fraction = (1 + count(calibration >= novelty(x)))
                / (1 + number_of_calibration_batches)

robust_z = abs(x - train_median) / (1.4826 * train_MAD)
```

A tail fraction at or below 0.10 triggers a novelty alert. The robust-MAD baseline alerts when any score exceeds 6; zero MAD is replaced by 1. Thresholds are fixed before the held-out benchmark. The calibration rank is descriptive, not a probability of corruption: seasonality, drift, and temporal dependence violate simple exchangeability assumptions.

Structural contracts

Required schema; unique source-row IDs; delivered-versus-ingested ID sets; exact field fidelity for matching IDs; dimension-key uniqueness; successful SQL execution; fact-row conservation; and fact-key uniqueness. These contracts explain a structural defect more directly than an unsupervised score.

In simple terms: the model asks whether the batch looks unusual. Contracts ask whether the pipeline obeyed its promises. A repair needs the second kind of evidence, not just the first.

A guarded hybrid investigator

Two explicit modes

Offline mode is a deterministic evidence policy running in LangGraph. It is the reproducible baseline and internet-free interview mode. Live mode calls a real local Ollama model. It chooses probe order and proposes structured diagnoses/actions; live-provider failures do not silently fall back to the baseline.

Read-only probe	Evidence returned
inspect_delivery	Schema differences, duplicated/missing IDs, changed-field counts, price scale ratios.
inspect_dimension	Row count, unique product keys, duplicate and conflicting key counts.
inspect_transform	Executed and approved SQL, change flag, row-preservation contract.

JSON schema permits only the named tools. All evidence probes must complete before finishing. At completion, deterministic evidence authorization restricts candidate roots and actions; the final diagnosis must reconcile with the evidence. This is a constrained hybrid agent, not a measurement of independent LLM root-cause reasoning. Model drafts are retained separately; the UI summary comes from verified evidence.

Repair catalog

- Deduplicate ingestion by source-row ID, refusing conflicting repeated values.

- Replay the immutable source partition for missing or altered fields/schema.

- Remove exact repeated dimension rows; conflicting descriptions require review.

- Restore the versioned row-preserving SQL transformation.

The model never executes arbitrary SQL or Python. Candidate repairs run on copied data. A trusted evaluator rebuilds the intended output from the source and approved SQL, then compares every output field, row identity, and multiplicity using an order-independent fingerprint. Matching revenue alone is insufficient.

Approval, rollback, and failure containment

Every run receives a UUID and separate original/verified Parquet snapshots. SQLite persists requests, results, status, and append-only application audit events. Events are durable application records, not cryptographically tamper-proof audit logs.

```
queued -> running -> needs_review -> approved -> rolled_back
                  -> no_action
                  -> manual_review / repair_rejected
                  -> investigation_failed / failed
server restart: queued or running -> interrupted
```

Approval rechecks the verified artifact digest and atomically changes the run-local active snapshot pointer. Repeated approval is idempotent. Rollback restores the original pointer. This does not deploy changes into a production warehouse or fix upstream application code globally.

Implemented controls

- Bounded queue, two workers, input validation, UUID path validation, idempotency keys, and explicit invalid-transition responses.

- Localhost host checks, same-origin mutation checks, content security policy, escaped UI text, request IDs, and optional API-client bearer authentication.

- Provider connect/read timeouts, bounded decisions, restricted tools/actions, independent output checks, and no hidden offline fallback.

- Source data stays unchanged; run isolation is logical data isolation, not an arbitrary-code sandbox.

Deployment boundary

The default is a single-user, single-process local application. Use one Uvicorn process because the queue is in memory. Completed runs persist; mid-tool resumption is not implemented. The UI has no login workflow. Public deployment requires identity/RBAC, TLS, durable distributed jobs, managed storage, secrets, retention, load tests, and organization-specific source connectors.

Container verification: Blocked by Docker environment timeout. The local Python app and tests pass, but Docker did not complete within its verification deadline. Container execution is not claimed as verified.

What the benchmark actually shows

The deterministic benchmark contains 120 cases: 12 held-out batch dates multiplied by ten scenarios. Seeds vary by date and severities cycle through 12%, 37%, and 63%. Eight scenario families are structural faults; healthy data and genuine source changes are negative controls.

Detector	Precision	Recall	F1	FP
Explicit source + transformation contracts	1.000	1.000	1.000	0
Isolation Forest novelty	0.762	0.167	0.274	5
Robust median absolute deviation	0.750	0.250	0.375	8
Contracts + Isolation Forest	0.950	1.000	0.975	5

Exact root-set accuracy was 100%; 96/96 faulty batches passed full-output repair verification; no unnecessary repair was proposed for 24 negative controls. Median offline evaluation time was 47.2 ms in the recorded run; this excludes browser/network overhead and is not a production latency guarantee.

Interpretation

Strong contracts detect all injected structural faults in this catalog. Isolation Forest misses defects that do not make raw-batch features unusual, especially downstream join and SQL failures. It also alerts on valid source changes. Combining novelty with contracts increases alerts and false positives for the declared corruption target; it does not improve this benchmark over contracts alone.

Threats to validity

- Known failure families and a trusted source/reference make diagnosis and verification much easier than open-ended incident response.

- Dates/seeds are held out, but failure types are not. Multiple cases share a date and are correlated; 120 cases are not 120 independent real-world incidents.

- The real source may contain errors that source-fidelity checks cannot discover. No business-owner ground truth exists for all source values.

- These scores belong to the deterministic baseline. Live mode uses evidence-constrained candidates and requires separate reporting.

Test evidence and live-model behavior

The automated suite currently passes 53 of 53 tests, with 94.6% statement coverage. Coverage is a code-execution measure, not proof that all behaviors are correct. Tests and benchmarks execute actual transformations and compare materialized output.

Important tests

- All ten scenario families; returns, null customer IDs, and legitimate duplicate business rows preserved.

- Conflicting IDs/dimension values rejected; changes in values caught even when row counts match.

- Provider timeout, premature completion, repeated tools, unsupported repairs, and prompt-like text in source fields.

- API validation, cross-origin writes, token checks, idempotency conflict, queue saturation, and unknown paths.

- Approval, repeated approval, rollback, altered verified artifacts, restart interruption, and concurrent run isolation.

Browser verification

The actual local UI was exercised for investigation, evidence inspection, approval, rollback, history reopening, negative-control behavior, and evaluation. Desktop 1440 x 1000 and mobile 390 x 844 were inspected; mobile document width was verified equal to the viewport after an overflow fix. This is not an exhaustive browser/device certification.

Live integration sample

join_fanout: needs_review (62.5s); compound: needs_review (44.2s); genuine_shift: no_action (41.1s)

Live tests use Qwen2.5:7b through Ollama and are separate from the offline 120-case result. A small sample establishes that the provider/tool path executes; it cannot establish general accuracy. Earlier small-model failures motivated stricter structured outputs, evidence-constrained action choices, and independent diagnosis checks. Preserve that lesson in the interview: a plausible explanation is not verification.

Evidence files: artifacts/tests.xml, coverage.json, benchmark.json, live-smoke.json, browser-qa.json, and packaging.json. The report generator reads these files rather than inventing performance numbers.

Explain the system in six minutes

Time	Demonstration
0:00-0:45	Explain silent data corruption: a successful SQL query can publish wrong totals.
0:45-1:30	Show the real source, preserve returns/nulls, explain controlled fault injection.
1:30-3:00	Run join fan-out. Inspect dimension cardinality, source fidelity, and unchanged SQL.
3:00-4:00	Show verified repair, approve it, roll back, and inspect audit events.
4:00-5:00	Run genuine source change: unusual data does not justify a repair.
5:00-6:00	Discuss detector comparison, offline/live distinction, and deployment limits.

A concrete example

Use October 5, 2011, seed 42, 30% affected fraction, join fan-out. The original batch has 2,447 rows and GBP 75,244.43 net revenue. Duplicate dimension keys inflate the output to 3,183 rows and GBP 99,330.93. Removing only identical repeated dimension records restores the full expected output. The verifier checks all values, not only those two totals.

Technical answers

Why not just SQL tests? They are a strong baseline and remain central. The added system coordinates investigation, records evidence, and stages verified changes. Why not fix all anomalies? Legitimate business changes can look unusual. Why an immutable landing area? It supplies provenance for reconciliation and replay. Why a small repair catalog? It creates an auditable action boundary instead of trusting generated code.

Research extension

The next defensible study would add independently authored unknown faults, multiple datasets, weaker source manifests, more LLMs, and repeated live runs with cost/latency measurements. Compare evidence access and repair policies under matched budgets. Add real incident labels before claiming production root-cause generalization or reduced mean time to recovery.

Start the app with make run and open http://127.0.0.1:8765. Do not open index.html directly: it needs the local API. Use Offline mode for a predictable interview, and identify it honestly as the deterministic baseline.

Three points and one resume bullet

Three points about the project

- Combined real transaction data, temporal anomaly modeling, SQL contracts, and a LangGraph investigation workflow to study silent pipeline corruption.

- Built an end-to-end review loop with read-only evidence probes, bounded repair recipes, isolated snapshots, full-output verification, approval, rollback, and a working UI.

- Evaluated known failure families reproducibly and reported the limits honestly: strong source contracts outperform statistical novelty on these structural faults, and live-model behavior is assessed separately.

One Google XYZ resume bullet

Engineered a data-pipeline reliability workbench that restored 96/96 injected faulty batches with 0 unnecessary repairs across 120 controlled cases, using Python, DuckDB, LangGraph, source-fidelity checks, and sandbox verification on 541K UCI retail records.

X = engineered the reliability workbench and restored faulty outputs. Y = 96/96 injected faults, zero unnecessary repairs, 120 controlled cases. Z = Python, DuckDB, LangGraph, source-fidelity contracts, and sandbox verification. Keep the words injected and controlled; do not present these numbers as production incidents or LLM-only accuracy.